✅ Task 1 — Basic Client–Server (Ultra-Clear Version)

Your goal in **Task 1** is to implement a very simple **TCP client and TCP server** that can send and receive messages in a specific format. You are **not** doing JSON, multiple clients, or logging yet — that comes later.
This task is worth **5 points**, so make sure you get this part correct.

🔵 1. Server Requirements (Very Important)

You must create a **TCP server** (`restaurant_server.py`) that:

✓ 1.1 Uses a hardcoded menu:

```
menu = {
    "PIZZ": {"name": "Margherita Pizza", "price": 1200, "stock": 10},
    "BURG": {"name": "Cheeseburger", "price": 800, "stock": 15},
    "SALA": {"name": "Caesar Salad", "price": 600, "stock": 20}
}
```

✓ 1.2 Waits for one client at a time (no concurrency needed yet)

✓ 1.3 Receives messages from the client in exactly this format:

```
action|item_id|quantity
```

Examples:

```
VIEW|PIZZ|0
ORDER|BURG|2
```

✓ 1.4 Parses the message and sends back the correct response

You **must** follow this response table exactly:

Situation	Server Response
VIEW success	`name:price:stock`
VIEW item doesn't exist	`NOT_FOUND`
ORDER success	`SUCCESS:total_price`
ORDER item doesn't exist	`NOT_FOUND`
ORDER insufficient stock	`NO_STOCK`
Invalid message format	`ERROR`

✓ 1.5 When an order succeeds

Reduce the stock:

Example:
Initial: stock = 10
ORDER|PIZZ|3 → SUCCESS:3600
New stock = 7

✓ 1.6 Handle client disconnects without crashing

🟢 2. What EXACTLY counts as a valid message?

A message is valid **only if**:

1. It contains **exactly 3 fields**
2. Fields are separated by `|`
3. `action` is either `VIEW` or `ORDER`
4. `item_id` is a **4-letter uppercase string** (e.g., “PIZZ”)
5. `quantity` is an integer
    - VIEW must send `0`
    - ORDER must send a positive number

If any of these fail → send `ERROR`

🔵 3. Client Requirements (student_client.py)

Your client must:

✓ 3.1 Connect to the server using TCP

Usage:

```
python3 student_client.py <ip> <port>
```

✓ 3.2 Repeatedly ask the user:

```
Enter action (VIEW/ORDER/QUIT):
Enter item ID:
Enter quantity (0 for VIEW):
```

✓ 3.3 Build a message in the correct format:

Example:

- VIEW → `VIEW|PIZZ|0`
- ORDER → `ORDER|PIZZ|3`

✓ 3.4 Send the message to the server

✓ 3.5 Receive the response and display it nicely

✓ 3.6 Continue until the user enters `QUIT`

At `QUIT`:
Close the connection and exit.

🟣 4. Simple Diagram (for clarity)

```
+--------------------+          +-----------------------+
|  student_client.py | <------> | restaurant_server.py  |
+--------------------+   TCP    +-----------------------+

Client sends:  VIEW|PIZZ|0
Server sends:  Margherita Pizza:1200:10

Client sends:  ORDER|PIZZ|3
Server sends:  SUCCESS:3600
(then server reduces stock)
```

🟩 5. Example of EXACT expected behaviour

Client:

```
Enter action (VIEW/ORDER/QUIT): VIEW
Enter item ID: PIZZ
Sent: VIEW|PIZZ|0
Response: Margherita Pizza:1200:10
```

Server:

```
Received: VIEW|PIZZ|0
Sending: Margherita Pizza:1200:10
```

Client:

```
Enter action: ORDER
Enter item ID: PIZZ
Enter quantity: 3
Sent: ORDER|PIZZ|3
Response: SUCCESS:3600
```

Server:

```
Received: ORDER|PIZZ|3
Sending: SUCCESS:3600
Updated stock: 7
```

This is **exactly** what students must reproduce.

🟧 6. Summary Checklist for Students

✓ TCP server
✓ Hardcoded menu
✓ Correct message format
✓ Correct response format
✓ Stock decreases on order
✓ Client keeps connection open
✓ QUIT ends the client
✓ No JSON, no logging, no UDP yet